***Task 1*** *(answer at least 10 out of 12)*
**After reviewing the Google File System papers, provide an answer to the following questions.**

**Q1.1** GFS uses *HeartBeat* for Master Check Server communication. Why are rather than a synchronous request-response interaction? What information is transferred via messages, and how is it used?

The Google File System uses periodic *HeartBeat* messages for communication between the master and chunkservers to minimize the overhead and improve the overall efficiency. Rather than relying on synchronous request-response interactions for every system event, HeartBeats allow GFS to merge routine tasks, such as storage reclamation, into the regular background activities of the master.

By batching this communication, the cost of the operations is amortised, and the master can process these updates when it is relatively free. This ensures the master can respond more promptly to urgent client requests that demand immediate attention.

During HearBeat exchanges, the following information is transferred and utilised:

- Chunkserver State and Inventories: Each chunkserver regularly reports its operational state and a subset of the chunks it currently holds on its disks
- Garbage Collection Directives: The master evaluates the reported chunks against its own file-to-chunk mappings. It replies via the HeartBeat with the identities of any chunks that are no longer present in its metadata, which authorises the chunkserver to delete those orphaned replicas to reclaim storage
- Lease Extensions: When a chunk is actively being mutated, the primary replica must maintain a lease. The primary's requests for lease extensions, as well as the master's grants of those extensions, are efficiently piggybacked onto the HeartBeat messages
- General Instructions and Monitoring: The master uses these regular messages to issue broader instructions to the chunkservers and to continuously monitor their status, ensuring it maintains an up-to-date view of the cluster's health without persistent querying

**Q1.2** What are the advantages of having a chunk size of 64 MB?

- A larger chunk sizes reduces the metadata significantly, this is important also because the metadata sits in the system memory on the master node
- It increases the likelihood that multiple operations will be performed on the same chunk
- Reduces the number of requests to locate a chunk
- Reduces disk fragmentation

**Q1.3** What is the technique used to reduce the interactions between the client and the master when a file is accessed in read mode?

The client will cache the chunk location, so the next time it can directly contact a chunkserver.

Also, using a large chunk size helps: it increases the likelihood that multiple operations will be performed on the same chunk. This increases the likelihood to end up on a chunk where the client already has the location in the cache, so it does not need to contact the master.

**Q1.4** How does the Master obtain the information about the chunk location?

This is achieved using two mechanisms:

1. During the chunkserver startup or during the commissioning phase, the chunkserver reports to the master all the chunks it holds
2. With HeartBeat messages: in heartbeat messages, a chunkserver reports a subset of the chunks it holds, the master replies with the identities of the chunks that are no longer present, so the chunkserver can delete them. This both allows garbage collection and allows the master to keep the location of the chunks up-to-date.

Initially, the Google File System designers attempted to keep chunk location information stored persistently on the master, but they decided it was much simpler to request the data directly from the chunkservers. This approach eliminates the complex problem of keeping the master and chunkservers synchronized as servers inevitably join, leave, fail, restart, or change names.

**Q1.5** Why is the Operation Log central to GFS, and how does the Master use the information stored in the Operation Log?

The operation log contains a historical record of critical metadata changes. It is central to GFS. Not only is it the only persistent record of metadata, but it also serves as a logical time line that defines the order of concurrent operations.

It is used for state recovery when master restarts or crashes: the checkpoint is restored and then the log file is replayed. Checkpoints are used to avoid very long operation logs, that would make the recovery phase take a lot of time.

**Q1.6** What is the meaning of regions for a file in GFS? When a file is in one of the above-mentioned states, how does the GFS recognize a given state?

In the Google File System (GFS), the contents of a file are composed of various data "regions," and the state of a file region is determined by the types of mutations (writes or appends) performed on it, whether they succeed or fail, and the presence of concurrent operations.
The regions of a file can fall into the following states:

- Consistent: A region is consistent if all clients will always see the exact same data, regardless of which chunk replica they happen to read from
- Defined: A region is defined if it is both consistent and allows clients to see the entirety of what the specific mutation wrote. This occurs after a successful mutation with no interference from concurrent writers
- Undefined (but consistent): When multiple concurrent mutations succeed, the region is left consistent (all clients see the same data), but it is undefined because it typically consists of

mingled data fragments from multiple writers, rather than reflecting one single mutation perfectly
- Inconsistent (and undefined): A failed mutation leaves a region inconsistent, meaning different clients might read different data at different times depending on the replica they access. Additionally, regions that contain padding or duplicate records, which GFS may insert during record appends, are considered inconsistent.

Under the hood, GFS ensures that regions are kept consistent and defined after successful mutations by applying those mutations in the exact same order across all chunk replicas, and by using chunk version numbers to proactively detect and isolate stale replicas.

**Q1.7** How does GFS maintain a consistent mutation order across replicas?

GFS maintains a consistent mutation order across replicas by utilizing a lease mechanism. The master grants a chunk lease to one of the replicas, designating it as the primary. This primary replica is responsible for assigning consecutive serial numbers to all incoming mutations, thereby establishing a strict serial order. All secondary replicas then apply the mutations in the exact same serial number order dictated by the primary, which ensures consistency across the system.

**Q1.8** What is the strategy used by GFS to transfer data to replicas, avoiding network bottlenecks and high-latency links, and to minimize latency?

GFS employs a strategy that decouples data flow from control flow, pushing data linearly across a chain of chunkservers rather than using a tree topology to fully utilize each machine's outbound bandwidth. To avoid network bottlenecks and high-latency links, each machine forwards the data to the "closest" chunkserver in the network topology—estimated using IP addresses—that has not yet received it. Finally, to minimize latency, GFS pipelines data transfers over TCP connections, allowing chunkservers to begin forwarding data immediately as they receive it over full-duplex links.

**Q1.9** Why is it not important that all the GFS replicas are byte-wise identical?

GFS does not require replicas to be byte-wise identical because its relaxed consistency model only guarantees that data is written *at least once* as an atomic unit. Due to retried operations or concurrent appends, individual replicas might contain varying padding or duplicate record fragments. This is not a problem because GFS applications are specifically designed to accommodate these variations.

**Q1.10** What are the goals of the Replica Placement policy used by GFS? How are the intended goals achieved?

The goals of the GFS chunk replica placement policy are to maximize data reliability and availability, and to maximize network bandwidth utilization.

These goals are achieved by spreading chunk replicas not just across different machines, but across multiple network racks. This multi-rack distribution ensures that some replicas of a chunk will survive and remain available even if an entire rack goes offline due to a shared resource failure, such as a power circuit or network switch. Additionally, spreading replicas across different racks allows data traffic—especially reads—to exploit the aggregate bandwidth of multiple racks, rather than being limited by the bandwidth of a single machine or rack.

**Q1.11** How did the master choose where to place a newly created chunk? When and why does the muster re-replicate a chunk?

When placing a newly created chunk, the master considers three main factors:

- Disk space equalization: It favors chunkservers that have below-average disk space utilization.
- Traffic management: It limits the number of recent chunk creations on any single chunkserver to prevent the server from being overwhelmed by the heavy write traffic that typically follows creation.
- Rack diversity: It spreads the chunk's replicas across multiple racks to ensure reliability and maximize aggregate network bandwidth.

The master re-replicates a chunk as soon as the number of available replicas falls below a user-specified goal,. This happens for a few common reasons:

- A chunkserver goes offline or becomes unavailable.
- A chunkserver reports that its replica may be corrupted.
- A chunkserver's disk is disabled due to errors.
- The replication goal is intentionally increased.

**Q1.12** How is the master state replicated for reliability? Why are " shadow " master replicas preferred to “mirror” replicas?

The master state is replicated by saving its operation log and checkpoints across multiple machines. A mutation is only considered committed after its log record is flushed to disk both locally and on all master replicas.

"Shadow" master replicas are preferred over strict "mirrors" because they provide read-only access to the file system to enhance availability, even if they lag behind the primary master by fractions of a second. To stay updated, a shadow master reads a replica of the growing operation log and applies the changes exactly as the primary does. This slight lag is acceptable because it only affects file

metadata; since applications read actual file content directly from the chunkservers, they do not observe stale file data.

***Task 2*** *(answer at least 10 out of 13)*
**After reviewing the Hadoop File System papers, provide an answer to the following questions.**

**Q2.1** Explain the concept of *image*, *checkpoint,* and *journal* in HDFS?

The image  is the metadata structure (kept entirely in RAM by the NameNode) that describes the hierarchical organization of the file system data, including inodes, directories, and files.
Otherwise the checkpoint is the persistent, on-disk record of the image stored in the NameNode's local file system. The NameNode never actively alters this file; instead, it is replaced in its entirety whenever a new checkpoint is created and the journal is a write-ahead commit log stored on disk that records all transactions and modifications to the metadata. Changes must be recorded and flushed to this log before they are committed to the clients to ensure persistence.

**Q2.2**. During normal operating conditions, DataNode sends *heartbeat* messages to the NameNode. Which information these heartbeats carry? How does the NameNode communicate with the DataNode?

The heartbeats carry statistics regarding the DataNode's total storage capacity, the fraction of storage currently in use, and the number of data transfers currently in progress.
The NameNode never calls DataNodes directly. Instead, it uses the replies it sends back to the DataNode heartbeats to piggyback instructions, such as commands to replicate or remove blocks

**Q2.3** When and why do DataNode and NameNode perform a ? What happens after a handshake?

A handshake is performed during the DataNode's startup. Its purpose is to verify that the DataNode's namespace ID and software version match the NameNode's, preventing data corruption or loss from incompatible nodes joining the cluster.
After a successful handshake, the DataNode registers its unique storage ID with the NameNode. Immediately after registration, it sends a *block report* to identify all the block replicas it currently possesses

**Q2.4** Can HDFS store and handle files with different replication factors? (E.g., file1 with replication factor 3 and file2 with replication factor 5)

Yes, because the HDFS is designed so that the replication factor is user-selectable on a file-by-file basis. While the default replication factor is three, an application can explicitly set a higher or lower replication factor for individual files depending on how critical or frequently accessed they are.

**Q2.5** How can a client application verify if a data block is corrupted? Is data corruption verified only by the client at read time?

The HDFS client verifies data by computing the checksum of the data it receives and comparing it against the original checksums shipped alongside the block. If they do not match, the client notifies the NameNode and fetches a different replica.
In addition to client-side checks, each DataNode runs a background process called a *block scanner*. This scanner periodically verifies all its stored block replicas against their associated checksums to proactively detect disk corruption

**Q2.6** What are the recommended practices for storing the checkpoint and journal?

To protect critical file system metadata from being lost or corrupted, the recommended practice is to configure HDFS to store the checkpoint and journal in multiple storage directories. Specifically, they should be placed on **different disk volumes** to prevent loss from a single volume failure, and one of those storage directories should be located on a **remote NFS server** to protect against the failure of the entire NameNode host

**Q2.7** Why does the CheckpointNode periodically combine the existing checkpoint and journal to create a new checkpoint and an empty journal?

The CheckpointNode periodically combines the existing checkpoint and journal to protect the file system metadata and prevent the journal file from growing indefinitely. Because HDFS clusters run for long periods without restarting, a constantly growing journal increases the probability of file corruption or loss, and drastically extends the time required to restart the NameNode. By creating a new checkpoint and an empty journal, the system can safely truncate the old journal and ensure quick recovery from the most recent checkpoint if other copies become unavailable.

**Q2.8** Does HDFS allow random writes on a file? (A random write is when a client application writes at a specified offset in a file)

No, HDFS does not allow random writes on a file. Once a file is written and closed, the existing bytes cannot be altered or removed. If a client application needs to modify a file, the only supported operation is reopening the file to append new data to the end.

**Q2.9** What are the steps performed by a client application and a DataNode for writing to a file? Can multiple clients write simultaneously on a file?

No, multiple clients cannot write to the same file simultaneously. HDFS enforces a single-writer, multiple-reader model by granting an exclusive lease to the client that opens a file for writing, which prevents any other client from modifying it.

The steps for writing to an HDFS file are:

1. Request DataNodes: The client asks the NameNode to allocate a new block and nominate a list of DataNodes to host its replicas.
2. Form a Pipeline: The chosen DataNodes are organized into a pipeline, ordered to minimize the network distance from the client.
3. Stream Packets: The client buffers its data into packets (typically 64 KB) and pushes them sequentially into the pipeline.
4. Acknowledge and Repeat: The packets flow from the client to the first DataNode, which forwards them to the second, and so on, with the DataNodes sending back acknowledgment messages. Once the block is filled, the client requests a new set of DataNodes from the NameNode to organize a new pipeline for the next block.

**Q2.10** What are the steps performed by a client application and a DataNode for reading a file? Can multiple clients read a file simultaneously? Are concurrent read and write operations on a file permitted?

The steps by a client application for reading a file in HDFS are

1. Fetch locations: The HDFS client contacts the NameNode to get the list of blocks for the file and the locations of their replicas, which are returned ordered by their distance from the reader.
2. Read data: The client directly contacts the closest DataNode to transfer the desired block.
3. Handle failures: If reading from the closest DataNode fails (e.g., node is unavailable or data is corrupt), the client tries the next closest replica in the sequence.

A file may have many concurrent readers simultaneously and also concurrent read and write operations are permitted: HDFS allows clients to read a file even while another client holds the lease and has it open for writing. Because the NameNode does not know the final length of the block still being written, the reading client will first ask one of the replicas for the latest length before starting to read.

**Q2.11** How are the DataNodes selected and ordered for a write operation? How are the DataNodes (replica locations) selected and ordered for a read operation?

When a client requests to write a new block, the NameNode selects DataNodes based on a placement policy designed to balance write cost, data reliability, and read bandwidth. By default,

the first replica is placed on the local node where the writer is located, the second and third replicas are placed on two different nodes within a different rack, and any additional replicas are placed randomly. Once selected, these DataNodes are organized into a pipeline in the order of their proximity (network distance) to the first replica, and data is pushed through this pipeline.
Instead, for a read operation, the NameNode returns the locations of the block replicas to the client, ordered by their network distance from the reader. The client then reads the data by trying the closest replica first; if that attempt fails (due to a node being unavailable or corrupt data), the client tries the next closest replica in the sequence.

**Q2.12** Describe the default HDFS block placement policy and the rationale behind the placement decisions.

The default HDFS block placement policy is structured as follows:

- First replica: Placed on the local node where the writer application is located.
- Second and third replicas: Placed on two different nodes within a single, different network rack.
- Additional replicas (if any): Placed randomly, with the restriction that no single node holds more than one replica, and no single rack holds more than two replicas (provided there are enough racks).

Rationale behind the placement decisions:

- Balanced Trade-off: The policy is designed to strike a balance between minimizing write costs and maximizing data reliability, availability, and aggregate read bandwidth.
- Optimized Write Performance: By placing the first replica locally and keeping the other two replicas on just one other rack, the system reduces costly inter-rack write traffic and significantly improves overall write performance.
- Fault Tolerance: Distributing replicas across two racks ensures that the data will survive even if an entire rack or network switch fails. Because full rack failures are much less common than individual node failures, this setup provides excellent data availability without the need to distribute replicas across three entirely different racks.
- Improved Read Bandwidth: Placing replicas on two unique racks instead of three reduces the aggregate network bandwidth consumed during read operations.

**Q2.13** How does HDFS guarantee that the disk utilization is balanced across the cluster Datanodes?

HDFS guarantees balanced disk utilization primarily through a specialized balancer tool that can be run by the cluster administrator. This tool iteratively moves block replicas from DataNodes with higher disk utilization to DataNodes with lower utilization until every node's usage is within a specific threshold compared to the overall cluster's average utilization.

Additionally, routine replica management naturally promotes balance: when a block becomes over-replicated and needs to be reduced, the NameNode prefers to remove the extra replica from the DataNode that has the least amount of available disk space.

***Task 3***

Discuss the similarities (and differences) between the GFS and HDFS with a focus on:

- Interactions among the client application, the node storing the data (data node and chunk server), and the master (or Name) node
- Communication strategy/technique used by the master/name node to interact with the nodes storing the data
- Data replication strategies and data placement strategies
- Access modes (In which modes client applications can write/read data)
- Data integrity, high availability, and recovery from failures

**1. Client, Data Node, and Master Interactions**

- **Similarities:** Both separate metadata from application data. Clients query the Master/NameNode for block locations, cache the metadata, and then perform I/O directly with the data nodes to prevent Master bottlenecks.
- **Differences: * GFS:** Decouples control and data flow. Clients push data linearly along a chain of chunkservers to maximize network bandwidth.
    - **HDFS:** Uses a sequential pipeline. The client pushes data as packets through a nominated suite of DataNodes, waiting for acknowledgments before closing.

**2. Master to Data Node Communication**

- **Similarities:** Both use regular **Heartbeats** to confirm node availability and transmit instructions. Neither Master permanently stores block locations on disk; they build in-memory maps by polling data nodes at startup.
- **Differences: * GFS:** Relies primarily on continuous heartbeats for status updates.
    - **HDFS:** Requires a strict startup handshake (verifying software/namespace IDs) and uses explicit, hourly **Block Reports** from DataNodes.

**3. Data Replication and Placement**

- **Similarities:** Both divide files into large blocks (64MB for GFS; typically 128MB for HDFS), default to 3 replicas, and use rack-aware placement to survive rack failures and maximize read bandwidth.
- **Differences: * GFS:** Balances disk usage organically in the background and limits active clone operations to prevent write hotspots.

    - **HDFS:** Uses a strict default placement (local node, remote rack node 1, remote rack node 2) and features a manual **Balancer tool** for administrators to iteratively fix skewed disk usage.

**4. Access Modes**

- **Similarities:** Both are optimized for Write-Once, Read-Many (WORM) workloads, large streaming reads, and sequential appends.
- **Differences: * GFS:** Supports concurrent writers via **Atomic Record Appends**, guaranteeing data is written at least once without distributed locks.
    - **HDFS:** Strictly enforces a **single-writer, multiple-reader** model by granting an exclusive write lease to a single client.

**5. Data Integrity, High Availability (HA), and Recovery**

- **Similarities:** Both use client-verified checksums for data integrity, replicate Master operation logs for HA, and rely on fast restarts for routine hardware failures.
- **Differences:**
    - **Integrity:** GFS uses chunk version numbers to catch stale replicas and checks data when idle. HDFS runs a continuous background **Block Scanner**.
    - **HA:** GFS uses read-only "shadow" masters. HDFS uses a Secondary/CheckpointNode (and modern automated standby nodes) to maintain namespace images.
    - **Recovery:** GFS relies on distributed logs and fast restarts. HDFS includes a **Snapshot** mechanism allowing admins to roll back the entire file system state if corrupted.